UniGe | MaLGa

Natural Language Processing

Language Modeling and Recurrent Neural Networks

Giacomo Meanti

November 12, 2026

Outline

Language modeling

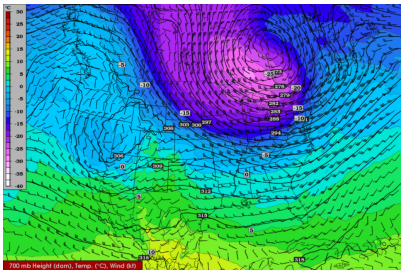
Classical approaches: n-grams

Neural language models

Recurrent neural networks

Some issues with RNNs

Language modeling



.



Language modeling

What is a language model?

- A mathematical model which can *describe* human language
- It should allow to *predict* and *evaluate* language

Language modeling

What is a language model?

- A mathematical model which can *describe* human language
- It should allow to *predict* and *evaluate* language

How can we create one?

- Language is ambiguous
- Language is evolving

Language modeling

What is a language model?

- A mathematical model which can *describe* human language
- It should allow to *predict* and *evaluate* language

How can we create one?

- Language is ambiguous
- Language is evolving

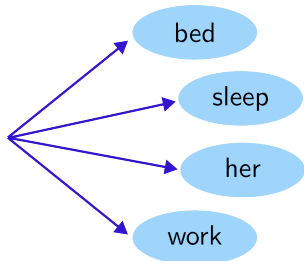
We will use *probabilistic models* of language

Language Modeling

Fundamental goal of a model is *prediction*

- Predict the next word

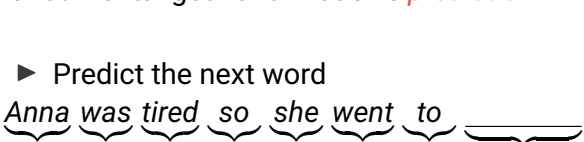
Anna $\boldsymbol{x}^{(1)}$ *was* $\boldsymbol{x}^{(2)}$ *tired* ... *so* *she* ... *went* $\boldsymbol{x}^{(7)}$ *to* $\boldsymbol{x}^{(8)}$ ____



Language Modeling

Fundamental goal of a model is *prediction*

- Predict the next word

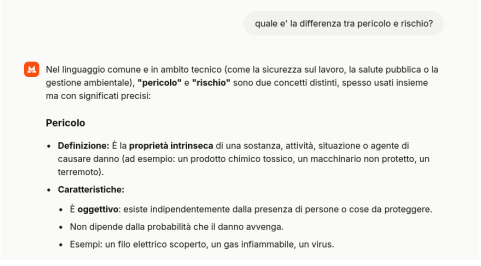


- Given a sentence $\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \ldots, \boldsymbol{x}^{(t)}$, compute the probability distribution of the next word $\boldsymbol{x}^{(t+1)}$

$$p(\boldsymbol{x}^{(t+1)} \mid \boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(1)})$$

Language Models in Daily Use





Language Modeling

An equivalent goal is to *evaluate* some text, or to assign it some probability

Given words $\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}$, the probability of this text is

$$p(\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}) = p(\boldsymbol{x}^{(1)}) \times p(\boldsymbol{x}^{(2)} \mid \boldsymbol{x}^{(1)}) \times \cdots \times p(\boldsymbol{x}^{(T)} \mid \boldsymbol{x}^{(T-1)}, \ldots, \boldsymbol{x}^{(1)})$$

$$= \prod_{t=1}^{T} \underbrace{p(\boldsymbol{x}^{(t)} \mid \boldsymbol{x}^{(t-1)}, \ldots, \boldsymbol{x}^{(1)})}_{\text{this is the next word prediction!}}$$

Recall: conditional probability $p(A, B) = p(A \mid B)p(B)$

Why care about language models?

Definition

A language model is a probabilistic model of natural language

- Language Modeling is a good benchmark for general NLP models
- Tasks which need to estimate the probability of text can be built on top of LMs
  - speech recognition
  - handwriting recognition
  - spell checking
  - machine translation
  - summarization
  - dialogue
  - etc.
- Everything else in NLP is built on top of language modeling: GPT is a LM!

Outline

Language modeling

Classical approaches: n-grams

Neural language models

Recurrent neural networks

Some issues with RNNs

n-gram language models

The quick brown fox jumps over the lazy _____

Definition (n-gram)

An n-gram is a sequence of n consecutive words

- unigrams : “the”, “quick”, “brown”, “fox”, etc.
- bigrams : “the quick”, “quick brown”, “brown fox”, “fox jumps”, etc.
- trigrams : “the quick brown”, “quick brown fox”, “brown fox jumps”, etc.
- four-grams : “the quick brown fox”, “quick brown fox jumps”, etc.

n-gram LMs:

- collect the frequencies of all n-grams in a text corpus
- use these frequencies to predict the next word

n-gram language models

Markov assumption

- In a **n**-gram model, assume $\boldsymbol{x}^{(t+1)}$ only depends on the $n-1$ previous words

$$p(\boldsymbol{x}^{(t+1)} \mid \boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(1)}) = p(\boldsymbol{x}^{(t+1)} \mid \underbrace{\boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(t-n+2)}}_{n-1 \text{ words}})$$

- For example with bigrams $p($"the quick brown fox"$)$ is decomposed as

$p($<s>$) \times p($"the" | <s>$) \times p($"quick" | "the"$) \times p($"brown" | "quick"$) \times p($"fox" | "brown"$) \times p($</s> | "fox"$)$

n-gram language models

Markov assumption

- In a **n**-gram model, assume $\boldsymbol{x}^{(t+1)}$ only depends on the $n-1$ previous words

$$p(\boldsymbol{x}^{(t+1)} \mid \boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(1)}) = p(\boldsymbol{x}^{(t+1)} \mid \underbrace{\boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(t-n+2)}}_{n-1 \text{ words}})$$

Conditional probability

$$p(\boldsymbol{x}^{(t+1)} \mid \boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(t-n+2)}) = \frac{p(\boldsymbol{x}^{(t+1)}, \boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(t-n+2)})}{p(\boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(t-n+2)})}$$

n-gram language models

Markov assumption

- In a **n**-gram model, assume $\boldsymbol{x}^{(t+1)}$ only depends on the $n-1$ previous words

$$p(\boldsymbol{x}^{(t+1)} \mid \boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(1)}) = p(\boldsymbol{x}^{(t+1)} \mid \underbrace{\boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(t-n+2)}}_{n-1 \text{ words}})$$

Conditional probability

$$p(\boldsymbol{x}^{(t+1)} \mid \boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(t-n+2)}) = \frac{p(\boldsymbol{x}^{(t+1)}, \boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(t-n+2)})}{p(\boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(t-n+2)})}$$

Finite-sample approximation

$$\approx \frac{\text{count}(\boldsymbol{x}^{(t+1)}, \boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(t-n+2)})}{\text{count}(\boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(t-n+2)})}$$

n-grams

$n-1$-grams

- Approximation quality depends on *size of text corpus*

n-gram language models: an example

Suppose we wish to learn a four-gram model

Anna was tired so she went to _____

n-gram language models: an example

Suppose we wish to learn a four-gram model

~~*Anna was tired so*~~ *she went to* _____

condition on this

$$p(\boldsymbol{x} \mid \text{“she went to”}) \approx \frac{\text{count}(\text{“she went to } \boldsymbol{x}\text{”})}{\text{count}(\text{“she went to”})}$$

Suppose in the corpus we have

- “she went to” with 1000 occurrences
- “she went to the” with 500 occurrences
- “she went to work” with 250 occurrences
- “she went to school” with 200 occurrences
- “she went to bed” with 50 occurrences

n-gram language models: an example

Suppose we wish to learn a four-gram model

~~*Anna was tired so*~~ *she went to* _____

condition on this

$$p(\boldsymbol{x} \mid \text{“she went to”}) \approx \frac{\text{count}(\text{“she went to } \boldsymbol{x}\text{”})}{\text{count}(\text{“she went to”})}$$

Suppose in the corpus we have

- “she went to” with 1000 occurrences
- “she went to the” with 500 occurrences
- “she went to work” with 250 occurrences
- “she went to school” with 200 occurrences
- “she went to bed” with 50 occurrences

Was it a good idea to discard the longer context?

Sparsity problems with n-gram LMs

$$p(\boldsymbol{x} \mid \text{“she went to”}) = \frac{\text{count}(\text{“she went to } \boldsymbol{x}\text{”})}{\text{count}(\text{“she went to”})}$$

Sparsity problems with n-gram LMs

$$p(\boldsymbol{x} \mid \text{“she went to”}) = \frac{\text{count}(\text{“she went to } \boldsymbol{x}\text{”})}{\text{count}(\text{“she went to”})}$$

Sparsity problem 1

- For many $\boldsymbol{x}$, “she went to $\boldsymbol{x}$” will never occur in the corpus.
- Many words with probability 0, make the LM inflexible.

Sparsity problems with n-gram LMs

$$p(\boldsymbol{x} \mid \text{“she went to”}) = \frac{\text{count}(\text{“she went to } \boldsymbol{x}\text{”}) + \epsilon}{\text{count}(\text{“she went to”})}$$

Sparsity problem 1

- For many $\boldsymbol{x}$, “she went to $\boldsymbol{x}$” will never occur in the corpus.
- Many words with probability 0, make the LM inflexible.
- **Mitigation:** add a small value ϵ to the count of every $\boldsymbol{x} \in V$ (*smoothing*)

Sparsity problems with n-gram LMs

$$p(\boldsymbol{x} \mid \text{“she went to”}) = \frac{\text{count}(\text{“she went to } \boldsymbol{x}\text{”}) + \epsilon}{\text{count}(\text{“she went to”})}$$

Sparsity problem 1

- For many $\boldsymbol{x}$, “she went to $\boldsymbol{x}$” will never occur in the corpus.
- Many words with probability 0, make the LM inflexible.
- **Mitigation:** add a small value ϵ to the count of every $\boldsymbol{x} \in V$ (*smoothing*)

Sparsity problem 2

- What if “she went to” never appeared in the corpus?
- Then we cannot calculate the probability!

Sparsity problems with n-gram LMs

$$p(\boldsymbol{x} \mid \text{“she went ~~to~~”}) = \frac{\text{count}(\text{“she went ~~to~~ } \boldsymbol{x}\text{”}) + \epsilon}{\text{count}(\text{“she went ~~to~~”})}$$

Sparsity problem 1

- For many $\boldsymbol{x}$, “she went to $\boldsymbol{x}$” will never occur in the corpus.
- Many words with probability 0, make the LM inflexible.
- **Mitigation:** add a small value ϵ to the count of every $\boldsymbol{x} \in V$ (*smoothing*)

Sparsity problem 2

- What if “she went to” never appeared in the corpus?
- Then we cannot calculate the probability!
- **Mitigation:** condition on just “she went” instead (*backoff*).

Sparsity problems with n-gram LMs

$$p(\boldsymbol{x} \mid \text{“she went ~~to~~”}) = \frac{\text{count}(\text{“she went ~~to~~ } \boldsymbol{x}\text{”}) + \epsilon}{\text{count}(\text{“she went ~~to~~”})}$$

Sparsity problem 1

- For many $\boldsymbol{x}$, “she went to $\boldsymbol{x}$” will never occur in the corpus.
- Many words with probability 0, make the LM inflexible.
- **Mitigation:** add a small value ϵ to the count of every $\boldsymbol{x} \in V$ (*smoothing*)

Sparsity problem 2

- What if “she went to” never appeared in the corpus?
- Then we cannot calculate the probability!
- **Mitigation:** condition on just “she went” instead (*backoff*).

Increasing n makes sparsity problems worse!
In practice $n \leq 5$.

Sparsity problems: an example

Berkeley Restaurant Project dataset with 9332 sentences ($|V| = 1446$).

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

This is with bigrams! Worse with $n > 2$.

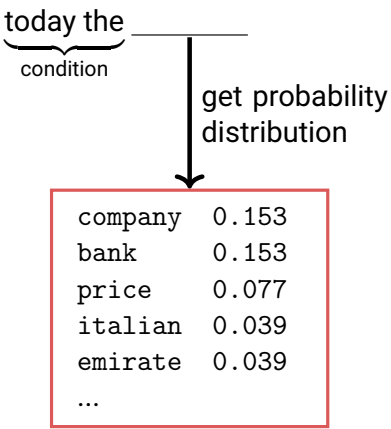
Storage problems with n-gram LMs

$$p(\boldsymbol{x} \mid \text{“she went to”}) = \frac{\text{count}(\text{“she went to } \boldsymbol{x}\text{”})}{\text{count}(\text{“she went to”})}$$

- Need to store counts for all n-grams seen in the corpus
- As n increases, or corpus size increases, this can become huge!
  If $|V| = 1446$ naively storing all 4-gram counts requires 4 TB of memory

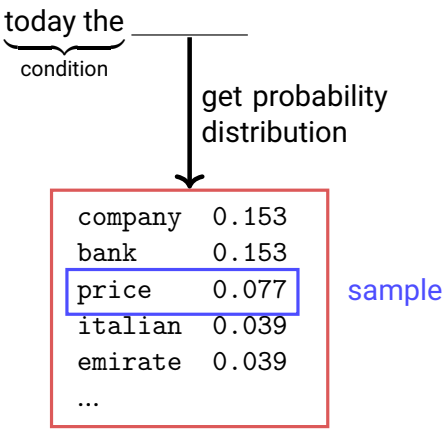
Practical example of n-gram LMs

- Trigram model, built on the Reuters word corpus (1.7M words, financial news)
- Generate text...



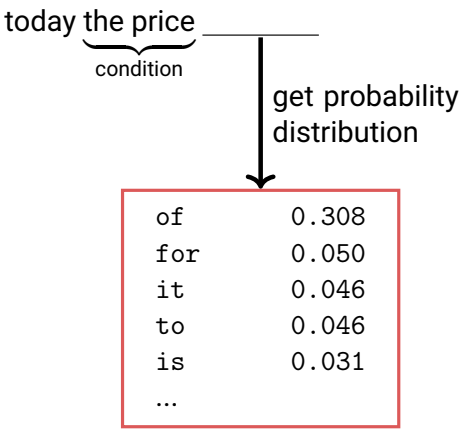
Practical example of n-gram LMs

- Trigram model, built on the Reuters word corpus (1.7M words, financial news)
- Generate text...



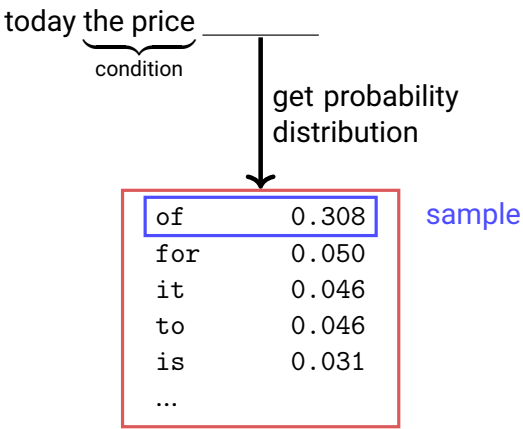
Practical example of n-gram LMs

- Trigram model, built on the Reuters word corpus (1.7M words, financial news)
- Generate text...



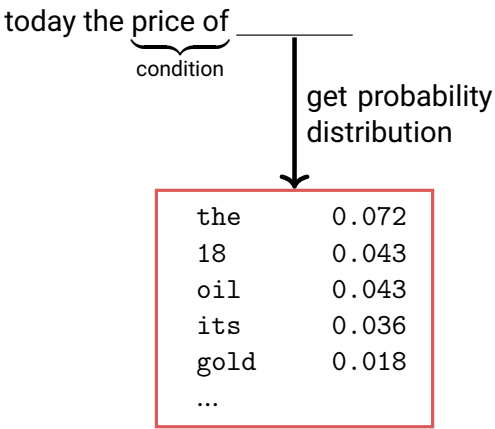
Practical example of n-gram LMs

- Trigram model, built on the Reuters word corpus (1.7M words, financial news)
- Generate text...



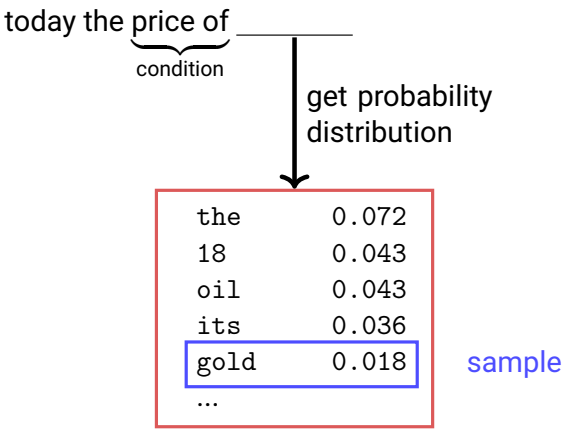
Practical example of n-gram LMs

- Trigram model, built on the Reuters word corpus (1.7M words, financial news)
- Generate text...



Practical example of n-gram LMs

- Trigram model, built on the Reuters word corpus (1.7M words, financial news)
- Generate text...



Text generation with n-gram LMs

n=1 “care I of . its destined , in , from . . for the Felix an not which measure excited”

n=2 “I compassionated him , how heavily ; but I had arrived , but they averred , my unhallowed damps and”

n=3 “I continued their single offspring . At length I gathered from a man who , born in freedom , spurned”

n=4 “I continued walking in this manner , during which I enjoyed the feeling of happiness . Still thou canst listen”

n=5 “I continued walking in this manner for some time , and I feared the effects of the dæmon’s disappointment .”

- Increasing n the model is more coherent.
- Increasing n increases sparsity problem.
- With high n the model repeats the training set!

https://towardsdatascience.com/text-generation-using-n-gram-model-8d12d9802aa0

Outline

Language modeling

Classical approaches: n-grams

Neural language models

Recurrent neural networks

Some issues with RNNs

Aside: Word Embeddings via Word2Vec

- Goal: represent each word with a vector. A word vector should be similar to other word vectors which occur in *similar* contexts. It should also be *dissimilar* to word vectors which are unlikely to appear in the same context.

$$\text{baking} = \begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \end{pmatrix} \quad \text{flour} = \begin{pmatrix} 0.413 \\ 0.582 \\ -0.007 \\ 0.247 \\ 0.216 \\ -0.718 \\ 0.147 \\ 0.051 \end{pmatrix}$$

$$\text{baking}^\top \text{flour} = 1.03$$

$$\text{baking} = \begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \end{pmatrix} \quad \text{dollar} = \begin{pmatrix} -0.154 \\ 0.024 \\ 0.512 \\ 0.782 \\ 0.359 \\ 0.141 \\ -0.581 \\ -0.027 \end{pmatrix}$$

$$\text{baking}^\top \text{dollar} = -0.35$$

Neural language models

Recall the LM task:

- Given a sequence of words $\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \ldots, \boldsymbol{x}^{(t)}$
- Output the probability distribution of next word $p(\boldsymbol{x}^{(t+1)} \mid \boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(t)})$

Using neural networks!

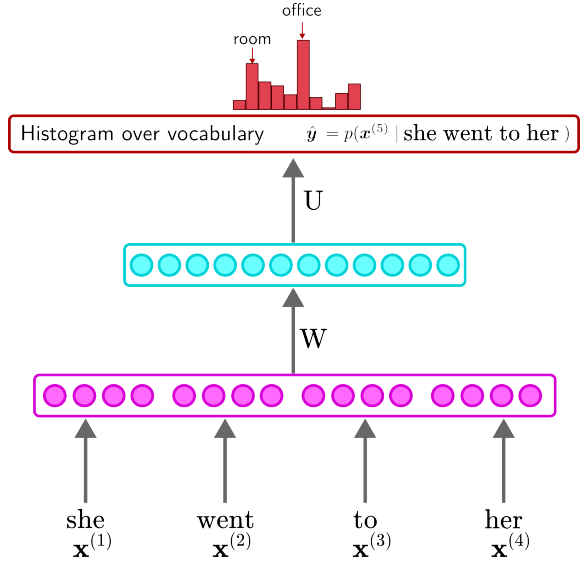
A fixed window approach (*n-grams + learning*)

~~Anna was tired so~~ she went to her _____

fixed window

- Predict missing word with a neural network

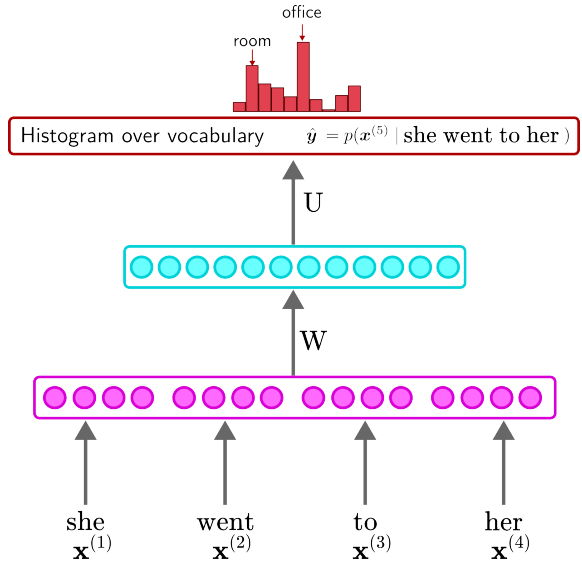
Fixed window neural LMs



one-hot word vectors

$\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \boldsymbol{x}^{(3)}, \boldsymbol{x}^{(4)}$

Fixed window neural LMs



word embeddings

$\boldsymbol{e} = [\boldsymbol{e}^{(1)}, \boldsymbol{e}^{(2)}, \boldsymbol{e}^{(3)}, \boldsymbol{e}^{(4)}]$

one-hot word vectors

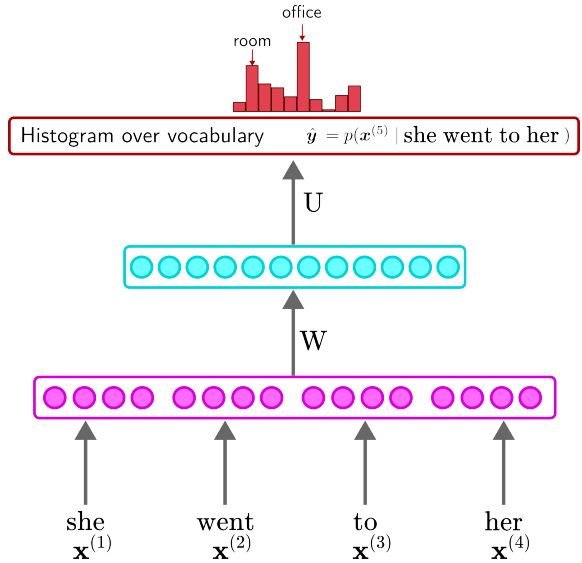
$\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \boldsymbol{x}^{(3)}, \boldsymbol{x}^{(4)}$

Fixed window neural LMs

hidden layer
$\boldsymbol{h} = \sigma(W_e \boldsymbol{e})$

word embeddings
$\boldsymbol{e} = [\boldsymbol{e}^{(1)}, \boldsymbol{e}^{(2)}, \boldsymbol{e}^{(3)}, \boldsymbol{e}^{(4)}]$

one-hot word vectors
$\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \boldsymbol{x}^{(3)}, \boldsymbol{x}^{(4)}$



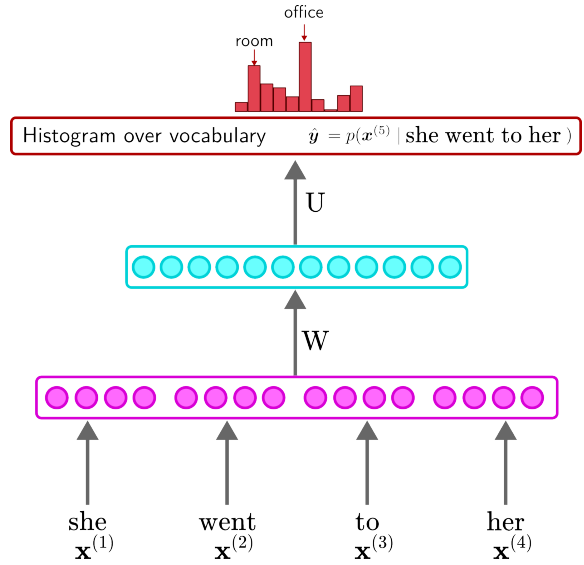
Fixed window neural LMs

output
$\hat{\boldsymbol{y}} = \text{softmax}(U\boldsymbol{h}) \in \mathbb{R}^{|V|}$

hidden layer
$\boldsymbol{h} = \sigma(W_e\boldsymbol{e})$

word embeddings
$\boldsymbol{e} = [\boldsymbol{e}^{(1)}, \boldsymbol{e}^{(2)}, \boldsymbol{e}^{(3)}, \boldsymbol{e}^{(4)}]$

one-hot word vectors
$\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \boldsymbol{x}^{(3)}, \boldsymbol{x}^{(4)}$



How to train a neural LM?

Definition (Training NNs)

1. Corpus of data
2. Model
3. Loss function J
4. Optimization algorithm

How to train a neural LM?

Definition (Training NNs)

1. Corpus of data $\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}$, a collection of documents
2. Model
3. Loss function J
4. Optimization algorithm

How to train a neural LM?

Definition (Training NNs)

1. Corpus of data $\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}$, a collection of documents
2. Model Fixed window neural LM
3. Loss function J
4. Optimization algorithm

How to train a neural LM?

Definition (Training NNs)

1. Corpus of data $\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}$, a collection of documents
2. Model Fixed window neural LM
3. Loss function J Compare output histogram to the *true probability distribution*
4. Optimization algorithm

How to train a neural LM?

Definition (Training NNs)

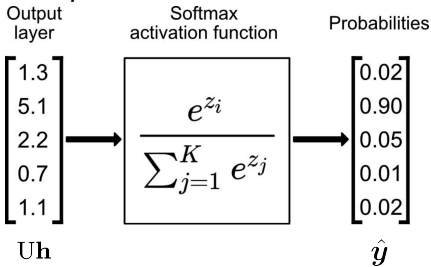
1. Corpus of data $\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}$, a collection of documents
2. Model Fixed window neural LM
3. Loss function J Compare output histogram to the *true probability distribution*
4. Optimization algorithm Stochastic gradient descent!

Loss function for LMs

- $Uh \in \mathbb{R}^{|V|}$ is *un-normalized prob. dist.*: Normalize it with the softmax

Output layer weights

Hidden layer encoding

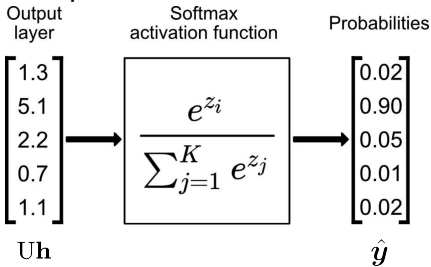


Loss function for LMs

- $\boldsymbol{U}\boldsymbol{h} \in \mathbb{R}^{|V|}$ is *un-normalized prob. dist.*: Normalize it with the softmax

Output layer weights

Hidden layer encoding



- Compare
  - computed probability $\widehat{\boldsymbol{y}}_i = q(V_i \mid \text{she went to her})$
  - true probability $\boldsymbol{y}_i^* = p(V_i \mid \textit{full context})$

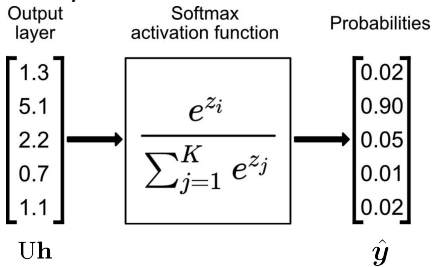
cross-entropy $\quad \text{CE}(p, \hat{\boldsymbol{y}}) = -\sum_{i=1}^{|V|} \boldsymbol{y}_i^* \log(\widehat{\boldsymbol{y}}_i)$

Loss function for LMs

- $Uh \in \mathbb{R}^{|V|}$ is *un-normalized prob. dist.*: Normalize it with the softmax

Output layer weights

Hidden layer encoding



- Compare
  - computed probability $\widehat{\boldsymbol{y}}_i = q(V_i \mid \text{she went to her})$
  - true probability $\boldsymbol{y}_i^* = p(V_i \mid \textit{full context})$

$$\textbf{cross-entropy} \quad \text{CE}(p, \hat{\boldsymbol{y}}) = -\sum_{i=1}^{|V|} \boldsymbol{y}_i^* \log(\widehat{\boldsymbol{y}}_i)$$

- But $\boldsymbol{y}_i^*$ is zero for every word different than $\boldsymbol{x}^{(5)}$ (the true word in the data):

$$\textbf{loss} \quad J^{(t)} = \text{CE}(p, \hat{\boldsymbol{y}}) = -\log(\hat{\boldsymbol{y}}_5) \quad \rightsquigarrow$$

negative log prob. of true class

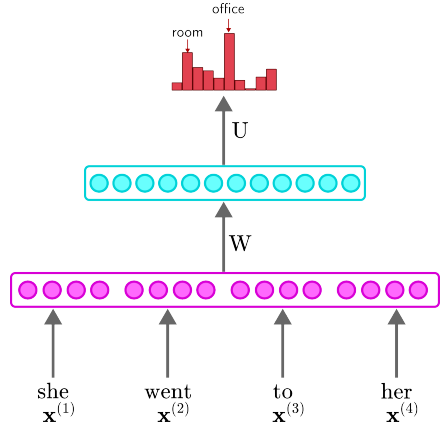
Fixed window neural LMs

Pros over n-gram LMs

- No sparsity problem (handled by word embeddings)
- No need to store n-grams

Remaining **problems**

- Fixed window is too restrictive
- Enlarging the window increases size of weights W
- No symmetry: $\boldsymbol{x}^{(1)}$ and $\boldsymbol{x}^{(2)}$ are all multiplied by different weights in W!



How can we process arbitrary-length inputs?

Y. Bengio et al. A Neural Probabilistic Language Model, 2003

Outline

Language modeling

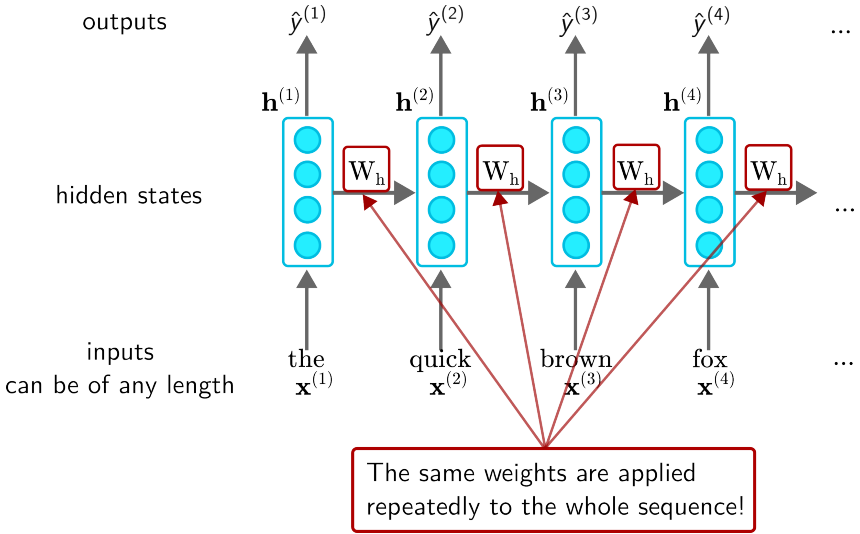
Classical approaches: n-grams

Neural language models

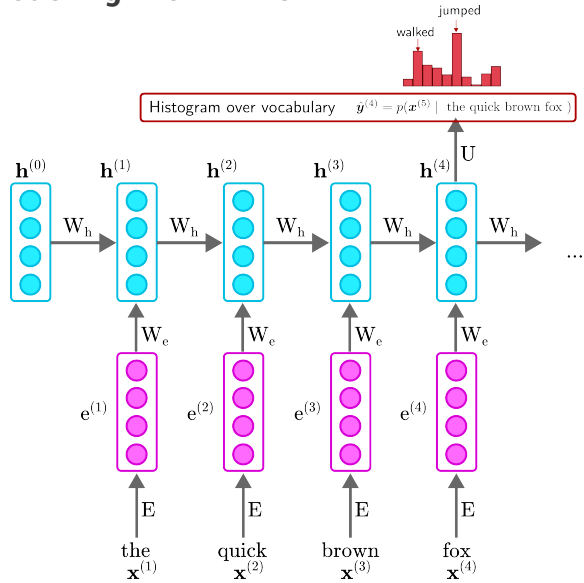
Recurrent neural networks

Some issues with RNNs

Recurrent Neural Networks (RNN)



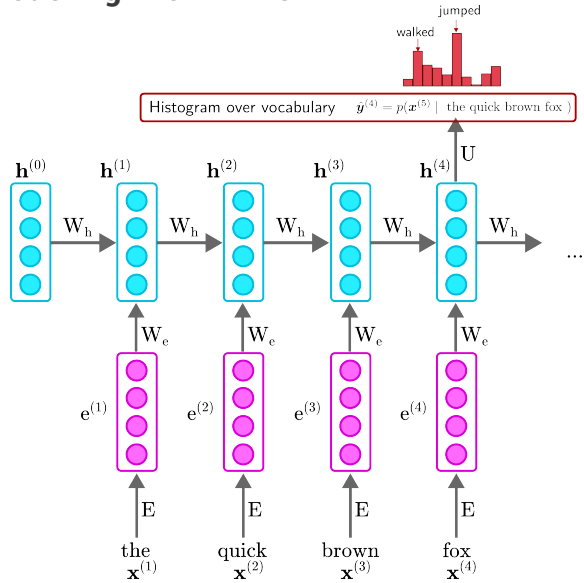
Language modeling with RNNs



one-hot word vectors

$\boldsymbol{x}^{(t)} \in \mathbb{R}^{|V|}$

Language modeling with RNNs



word embeddings

$\boldsymbol{e}^{(t)} = E\boldsymbol{x}^{(t)}$

one-hot word vectors

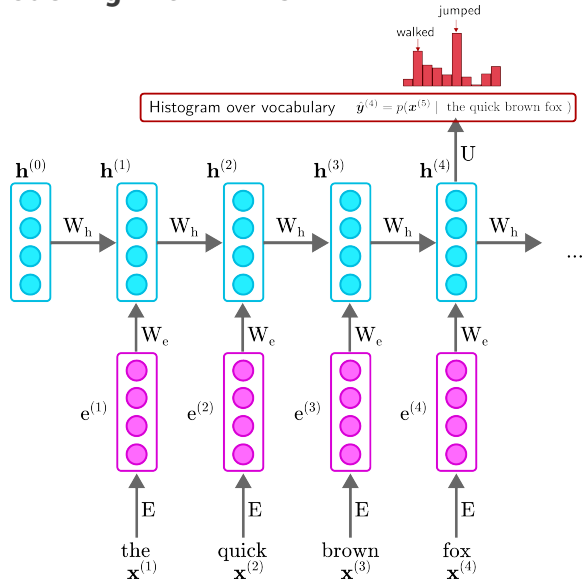
$\boldsymbol{x}^{(t)} \in \mathbb{R}^{|V|}$

Language modeling with RNNs

hidden states
$\boldsymbol{h}^{(t)} = \sigma(W_h \boldsymbol{h}^{(t-1)} + W_e \boldsymbol{e}^{(t)})$

word embeddings
$\boldsymbol{e}^{(t)} = E\boldsymbol{x}^{(t)}$

one-hot word vectors
$\boldsymbol{x}^{(t)} \in \mathbb{R}^{|V|}$



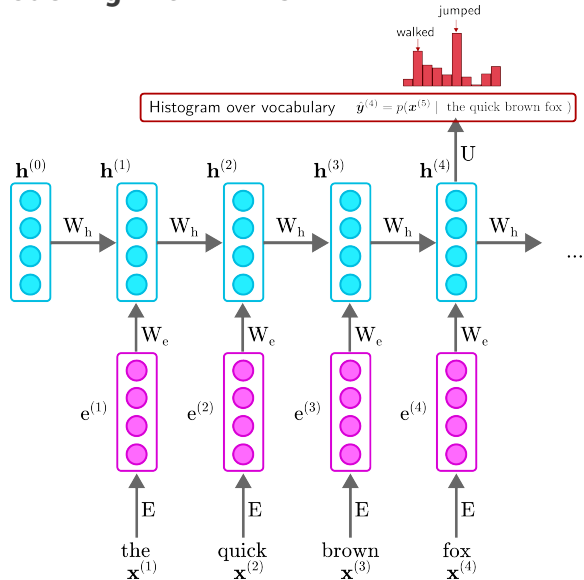
Language modeling with RNNs

output
$\hat{\boldsymbol{y}}^{(t)} = \text{softmax}(U\boldsymbol{h}^{(t)}) \in \mathbb{R}^{|V|}$

hidden states
$\boldsymbol{h}^{(t)} = \sigma(W_h\boldsymbol{h}^{(t-1)} + W_e\boldsymbol{e}^{(t)})$

word embeddings
$\boldsymbol{e}^{(t)} = E\boldsymbol{x}^{(t)}$

one-hot word vectors
$\boldsymbol{x}^{(t)} \in \mathbb{R}^{|V|}$



Language modeling with RNNs

Pros

- Not restricted to fixed-length inputs
- Can use information from many timesteps ago (in theory)
- Input words are processed symmetrically (*weight sharing*)

Cons

- Computation is slow (no parallelism)
- In practice long-range information is hard to capture

Training procedure for LMs

1. Gather a large corpus of text $\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}$

Training procedure for LMs

1. Gather a large corpus of text $\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}$
2. Give it to the RNN, computing $\hat{\boldsymbol{y}}^{(t)}$ for $t = 1, \ldots, T$

Training procedure for LMs

1. Gather a large corpus of text $\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}$
2. Give it to the RNN, computing $\hat{\boldsymbol{y}}^{(t)}$ for $t = 1, \ldots, T$
3. At each step, compute a loss function:
   cross-entropy between predicted probability distribution of next word $\hat{\boldsymbol{y}}^{(t)}$ and true next word $\boldsymbol{y}^{(t)} = \boldsymbol{x}^{(t+1)}$

$$J^{(t)} = \text{CE}(\boldsymbol{y}^{(t)}, \hat{\boldsymbol{y}}^{(t)}) = -\sum_{w \in V} \overbrace{\boldsymbol{y}_w^{(t)}}^{\text{one-hot}} \log \hat{\boldsymbol{y}}_w^{(t)} = -\log \hat{\boldsymbol{y}}_{x_{t+1}}^{(t)}$$

negative log probability of correct word

Training procedure for LMs

1. Gather a large corpus of text $\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}$
2. Give it to the RNN, computing $\hat{\boldsymbol{y}}^{(t)}$ for $t = 1, \ldots, T$
3. At each step, compute a loss function:
   cross-entropy between predicted probability distribution of next word $\hat{\boldsymbol{y}}^{(t)}$ and true next word $\boldsymbol{y}^{(t)} = \boldsymbol{x}^{(t+1)}$

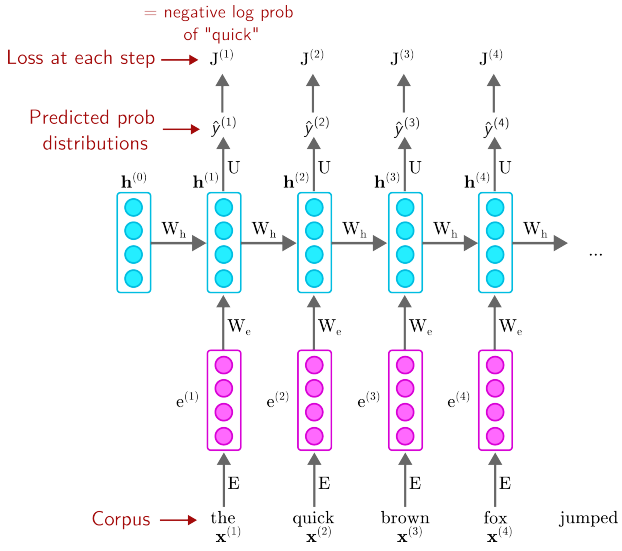
$$J^{(t)} = \text{CE}(\boldsymbol{y}^{(t)}, \hat{\boldsymbol{y}}^{(t)}) = -\sum_{w \in V} \overbrace{\boldsymbol{y}_w^{(t)}}^{\text{one-hot}} \log \hat{\boldsymbol{y}}_w^{(t)} = -\log \hat{\boldsymbol{y}}_{x_{t+1}}^{(t)}$$

negative log probability of correct word

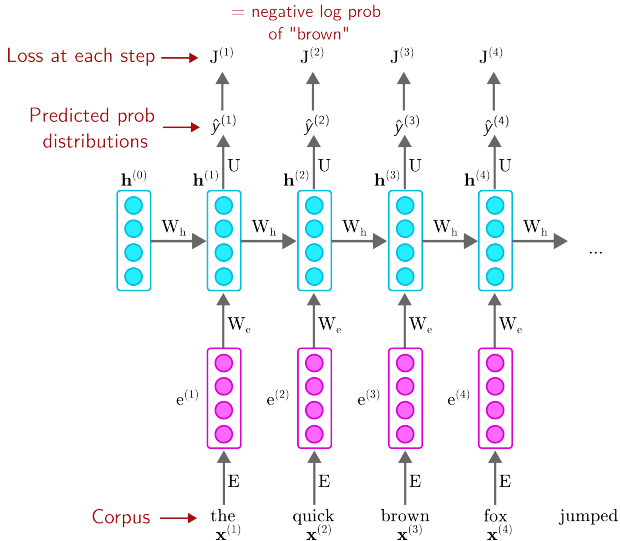
4. Average loss over the dataset

$$J = \frac{1}{T-1} \sum_{t=1}^{T-1} J^{(t)} = \frac{1}{T-1} \sum_{t=1}^{T-1} -\log \hat{\boldsymbol{y}}_{x_{t+1}}^{(t)}$$

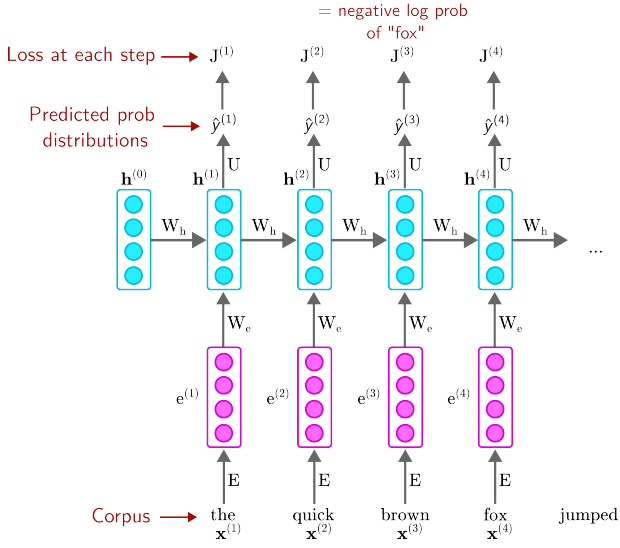
Training procedure for LMs



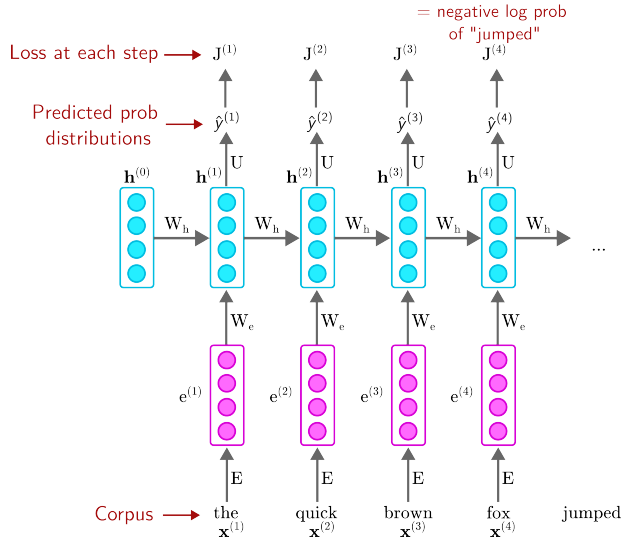
Training procedure for LMs



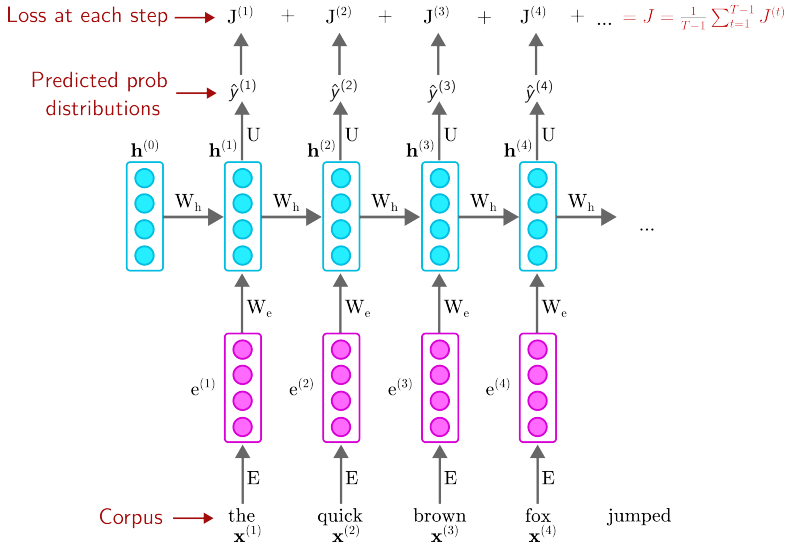
Training procedure for LMs



Training procedure for LMs



Training procedure for LMs



Training a RNN in practice

- Typical text corpus may have millions of words. Computing loss and gradients across all of it requires too much memory

$$J(\theta) = \frac{1}{T-1}\sum_{t=1}^{T-1} J^{(t)}(\theta)$$

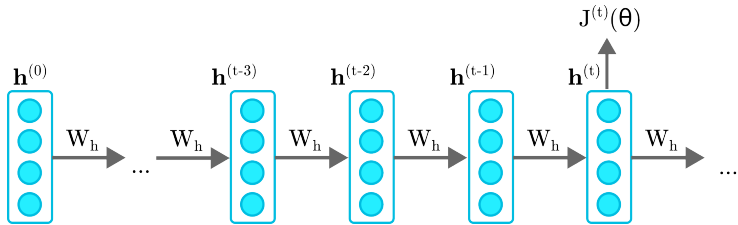
- Divide the corpus into smaller chunks (i.e. sentences, documents)
- Stochastic Gradient Descent gives us a way to approximate $\nabla_\theta J(\theta)$ on small chunks of data of size S

$$\nabla_\theta J(\theta) \approx \frac{1}{S}\sum_{j=1}^{S} \nabla_\theta J^{(j)}(\theta)$$

- Loss and gradients are computed for one *batch* of sentences at a time.

Backpropagation for RNNs

Even simpler model: $\boldsymbol{h}^{(t)} = W_h \boldsymbol{h}^{(t-1)}, \quad J^{(t)}(\theta) = \text{CE}(\boldsymbol{h}^{(t)}, y^{*(t)})$



What is the gradient of $J^{(t)}(\theta)$ w.r.t. the repeated weight matrix W_h?

$$\frac{\partial J^{(t)}}{\partial W_h} = \sum_{i=1}^{t} \frac{\partial J^{(t)}}{\partial W_h}\Big|_{(i)}$$

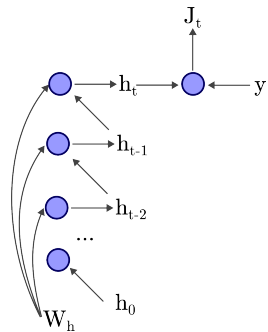
Backpropagation through time: gradient w.r.t. a repeated weight is the sum of the gradient for each time it appears.

Backpropagation through time (BPTT)

Simplified model:

$$\boldsymbol{h}^{(t)} = W_h \boldsymbol{h}^{(t-1)}, \quad J^{(t)}(\theta) = \mathsf{CE}(\boldsymbol{h}^{(t)}, y^{*(t)})$$

$$\frac{\partial J^{(t)}}{\partial W_h} =$$



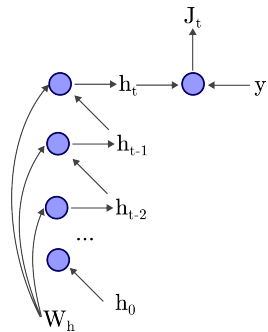
Backpropagation through time (BPTT)

Simplified model:

$$\boldsymbol{h}^{(t)} = W_h \boldsymbol{h}^{(t-1)}, \quad J^{(t)}(\theta) = \mathsf{CE}(\boldsymbol{h}^{(t)}, y^{*(t)})$$

$$\frac{\partial J^{(t)}}{\partial W_h} = \frac{\partial J^{(t)}}{\partial \boldsymbol{h}^{(t)}} \frac{\partial \boldsymbol{h}^{(t)}}{\partial W_h}$$

chain rule



Backpropagation through time (BPTT)

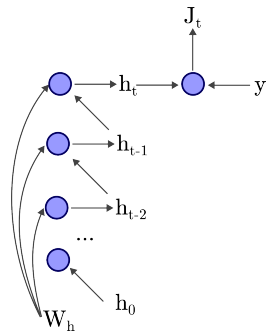
Simplified model:

$$\boldsymbol{h}^{(t)} = W_h \boldsymbol{h}^{(t-1)}, \quad J^{(t)}(\theta) = \mathsf{CE}(\boldsymbol{h}^{(t)}, y^{*(t)})$$

$$\frac{\partial J^{(t)}}{\partial W_h} = \frac{\partial J^{(t)}}{\partial \boldsymbol{h}^{(t)}} \frac{\partial \boldsymbol{h}^{(t)}}{\partial W_h}$$

chain rule

$\boldsymbol{h}^{(t)}$ depends on W_h directly and through $\boldsymbol{h}^{(t-1)}$. Call it $\boldsymbol{h}^{(t)}(W_h, \boldsymbol{h}^{(t-1)})$



Backpropagation through time (BPTT)

Simplified model:

$$\boldsymbol{h}^{(t)} = W_h \boldsymbol{h}^{(t-1)}, \quad J^{(t)}(\theta) = \mathrm{CE}(\boldsymbol{h}^{(t)}, y^{*(t)})$$

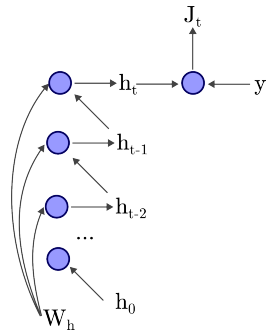
$$\frac{\partial J^{(t)}}{\partial W_h} = \frac{\partial J^{(t)}}{\partial \boldsymbol{h}^{(t)}} \frac{\partial \boldsymbol{h}^{(t)}}{\partial W_h}$$

chain rule

$\boldsymbol{h}^{(t)}$ depends on W_h directly and through $\boldsymbol{h}^{(t-1)}$. Call it $\boldsymbol{h}^{(t)}(W_h, \boldsymbol{h}^{(t-1)})$

$$\frac{\partial \boldsymbol{h}^{(t)}(W_h, \boldsymbol{h}^{(t-1)})}{\partial W_h} = \underbrace{\frac{\partial \boldsymbol{h}^{(t)}}{\partial W_h} \frac{\partial W_h}{\partial W_h}}_{\text{direct}} + \underbrace{\frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{h}^{(t-1)}} \frac{\partial \boldsymbol{h}^{(t-1)}}{\partial W_h}}_{\text{indirect}}$$

derivative of composition



Backpropagation through time (BPTT)

Simplified model:

$$\boldsymbol{h}^{(t)} = W_h \boldsymbol{h}^{(t-1)}, \quad J^{(t)}(\theta) = \mathrm{CE}(\boldsymbol{h}^{(t)}, y^{*(t)})$$

$$\frac{\partial J^{(t)}}{\partial W_h} = \frac{\partial J^{(t)}}{\partial \boldsymbol{h}^{(t)}} \frac{\partial \boldsymbol{h}^{(t)}}{\partial W_h}$$

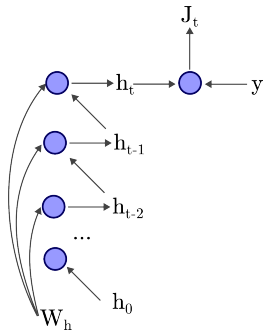
chain rule

$\boldsymbol{h}^{(t)}$ depends on W_h directly and through $\boldsymbol{h}^{(t-1)}$. Call it $\boldsymbol{h}^{(t)}(W_h, \boldsymbol{h}^{(t-1)})$

$$\frac{\partial \boldsymbol{h}^{(t)}(W_h, \boldsymbol{h}^{(t-1)})}{\partial W_h} = \underbrace{\frac{\partial \boldsymbol{h}^{(t)}}{\partial W_h} \frac{\partial W_h}{\partial W_h}}_{\text{direct}} + \underbrace{\frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{h}^{(t-1)}} \frac{\partial \boldsymbol{h}^{(t-1)}}{\partial W_h}}_{\text{indirect}}$$

derivative of composition

$$= \boldsymbol{h}^{(t-1)} I + W_h \left(\frac{\partial \boldsymbol{h}^{(t-1)}(W_h, \boldsymbol{h}^{(t-2)})}{\partial W_h} \right)$$



Backpropagation through time (BPTT)

Simplified model:

$$\boldsymbol{h}^{(t)} = W_h \boldsymbol{h}^{(t-1)}, \quad J^{(t)}(\theta) = \mathrm{CE}(\boldsymbol{h}^{(t)}, y^{*(t)})$$

$$\frac{\partial J^{(t)}}{\partial W_h} = \frac{\partial J^{(t)}}{\partial \boldsymbol{h}^{(t)}} \frac{\partial \boldsymbol{h}^{(t)}}{\partial W_h}$$

chain rule

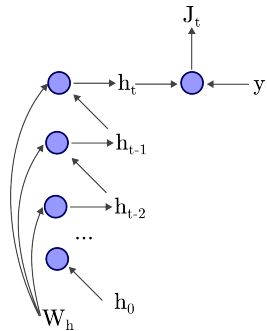
$\boldsymbol{h}^{(t)}$ depends on W_h directly and through $\boldsymbol{h}^{(t-1)}$. Call it $\boldsymbol{h}^{(t)}(W_h, \boldsymbol{h}^{(t-1)})$

$$\frac{\partial \boldsymbol{h}^{(t)}(W_h, \boldsymbol{h}^{(t-1)})}{\partial W_h} = \underbrace{\frac{\partial \boldsymbol{h}^{(t)}}{\partial W_h}\frac{\partial W_h}{\partial W_h}}_{\text{direct}} + \underbrace{\frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{h}^{(t-1)}}\frac{\partial \boldsymbol{h}^{(t-1)}}{\partial W_h}}_{\text{indirect}}$$

derivative of composition

$$= \boldsymbol{h}^{(t-1)} I + W_h \left(\frac{\partial \boldsymbol{h}^{(t-1)}(W_h, \boldsymbol{h}^{(t-2)})}{\partial W_h} \right)$$

$$= \boldsymbol{h}^{(t-1)} + W_h \left(\boldsymbol{h}^{(t-2)} + W_h \left(\frac{\partial \boldsymbol{h}^{(t-2)}(W_h, \boldsymbol{h}^{(t-3)})}{\partial W_h} \right) \right)$$



Backpropagation through time (BPTT)

Simplified model:

$$\boldsymbol{h}^{(t)} = W_h \boldsymbol{h}^{(t-1)}, \quad J^{(t)}(\theta) = \mathrm{CE}(\boldsymbol{h}^{(t)}, y^{*(t)})$$

$$\frac{\partial J^{(t)}}{\partial W_h} = \frac{\partial J^{(t)}}{\partial \boldsymbol{h}^{(t)}} \frac{\partial \boldsymbol{h}^{(t)}}{\partial W_h}$$

chain rule

$\boldsymbol{h}^{(t)}$ depends on W_h directly and through $\boldsymbol{h}^{(t-1)}$. Call it $\boldsymbol{h}^{(t)}(W_h, \boldsymbol{h}^{(t-1)})$

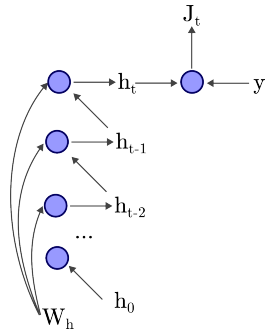
$$\frac{\partial \boldsymbol{h}^{(t)}(W_h, \boldsymbol{h}^{(t-1)})}{\partial W_h} = \underbrace{\frac{\partial \boldsymbol{h}^{(t)}}{\partial W_h}\frac{\partial W_h}{\partial W_h}}_{\text{direct}} + \underbrace{\frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{h}^{(t-1)}}\frac{\partial \boldsymbol{h}^{(t-1)}}{\partial W_h}}_{\text{indirect}}$$

derivative of composition

$$= \boldsymbol{h}^{(t-1)} I + W_h \left(\frac{\partial \boldsymbol{h}^{(t-1)}(W_h, \boldsymbol{h}^{(t-2)})}{\partial W_h} \right)$$

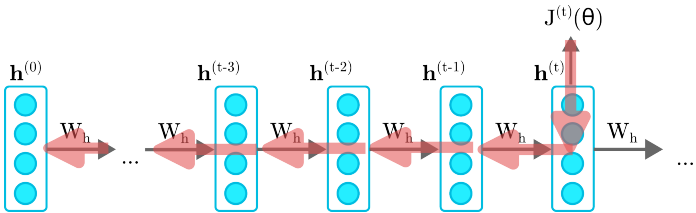
$$= \boldsymbol{h}^{(t-1)} + W_h \left(\boldsymbol{h}^{(t-2)} + W_h \left(\frac{\partial \boldsymbol{h}^{(t-2)}(W_h, \boldsymbol{h}^{(t-3)})}{\partial W_h} \right) \right)$$

$$= \cdots = \sum_{i=0}^{t-1} \left(\prod_{j=i+1}^{t-1} W_h \right) \boldsymbol{h}^{(i)}$$



Backpropagation through time (BPTT)

BPTT: equivalent as backpropagating through the whole network



$$\frac{\partial J^{(t)}}{\partial W_h} = \sum_{i=1}^{t} \frac{\partial J^{(t)}}{\partial W_h}\Big|_{(i)}$$

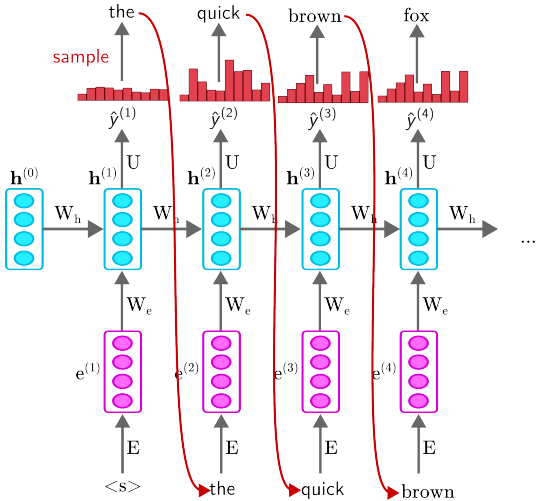
For more complex RNNs the formula is very similar!

In practice, truncate backward pass at $\sim$ 20 steps for efficiency

https://d2l.ai/chapter_recurrent-neural-networks/bptt.html

Text generation with an RNN

To generate text at test time, you can repeatedly sample from the network, feeding outputs back as inputs



Evaluating LMs: perplexity

- Take a *test sentence* $X_{\text{test}} = \boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}$ unseen during training!

Definition (Perplexity)

The inverse probability of the sentence, normalized by its length T

$$\text{perplexity}(X_{\text{test}}) = p(\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)})^{-\frac{1}{T}} = \sqrt[T]{\frac{1}{p(\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)})}}$$

Evaluating LMs: perplexity

- Take a *test sentence* $X_{\text{test}} = \boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}$ unseen during training!

Definition (Perplexity)

The inverse probability of the sentence, normalized by its length T

$$\text{perplexity}(X_{\text{test}}) = p(\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)})^{-\frac{1}{T}} = \sqrt[T]{\frac{1}{p(\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)})}}$$

1. Compute the probability using the tested language model and the chain rule

$$p(\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}) = \prod_{t=1}^{T} p(\boldsymbol{x}^{(t)} \mid \boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(t-1)})$$

Evaluating LMs: perplexity

- Take a *test sentence* $X_{\text{test}} = \boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}$ unseen during training!

Definition (Perplexity)

The inverse probability of the sentence, normalized by its length T

$$\text{perplexity}(X_{\text{test}}) = p(\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)})^{-\frac{1}{T}} = \sqrt[T]{\frac{1}{p(\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)})}}$$

1. Compute the probability using the tested language model and the chain rule

$$p(\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}) = \prod_{t=1}^{T} p(\boldsymbol{x}^{(t)} \mid \boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(t-1)})$$

2. compute $p(\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)})^{-\frac{1}{T}}$

Perplexity

- In practice computing $\prod_{t=1}^{T} p(\boldsymbol{x}^{(t)} \mid \boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(t-1)})$ for long sequences can be unstable.
  - Include BoS and EoS (`<s>`, `</s>`) markers
  - Truncate conditioning at sentence/document boundaries
- Connection to entropy:
  - $p(\boldsymbol{x}^{(t)} \mid \boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(t-1)}) = \hat{\boldsymbol{y}}^{(t)}_{\boldsymbol{x}^{(t+1)}}$

$$\text{perplexity}(X_{\text{test}}) = \left[\prod_{t=1}^{T} \hat{\boldsymbol{y}}^{(t)}_{\boldsymbol{x}^{(t+1)}}\right]^{-\frac{1}{T}} = \exp\left(\underbrace{\frac{1}{T}\sum_{i=1}^{T} -\log \hat{\boldsymbol{y}}^{(t)}_{\boldsymbol{x}^{(t+1)}}}_{\text{CE loss of the sequence}}\right)$$

- Lower perplexity: the model is less “surprised” by provided text

Perplexity: an example

vocabulary of numbers from 0 to 9, $|V| = 10$.

1. LM1: outputs any number with probability $\frac{1}{10}$
2. LM2: outputs 0 with probability $\frac{91}{100}$, other numbers with equal probability $\frac{1}{100}$

Given sequence “0 0 0 3 0”, which LM has lowest perplexity?

Perplexity: an example

vocabulary of numbers from 0 to 9, $|V| = 10$.

1. LM1: outputs any number with probability $\frac{1}{10}$
2. LM2: outputs 0 with probability $\frac{91}{100}$, other numbers with equal probability $\frac{1}{100}$

Given sequence “0 0 0 3 0”, which LM has lowest perplexity?

LM1

$$\text{perplexity} = \left(\frac{1}{10}^{5}\right)^{-\frac{1}{5}} = \frac{1}{10}^{-1} = 10$$

Perplexity: an example

vocabulary of numbers from 0 to 9, $|V| = 10$.

1. LM1: outputs any number with probability $\frac{1}{10}$
2. LM2: outputs 0 with probability $\frac{91}{100}$, other numbers with equal probability $\frac{1}{100}$

Given sequence “0 0 0 3 0”, which LM has lowest perplexity?

LM1

$$\text{perplexity} = \left(\frac{1}{10}^{5}\right)^{-\frac{1}{5}} = \frac{1}{10}^{-1} = 10$$

LM2

$$\text{perplexity} = \left(\frac{91}{100}^{4} \times \frac{1}{100}\right)^{-\frac{1}{5}} = 2.709$$

Outline

Language modeling

Classical approaches: n-grams

Neural language models

Recurrent neural networks

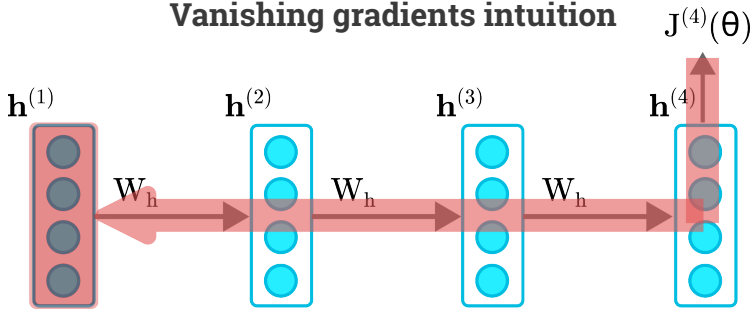
Some issues with RNNs

Problems with RNNs

In practice training vanilla RNNs (the model presented so far) is *hard*:

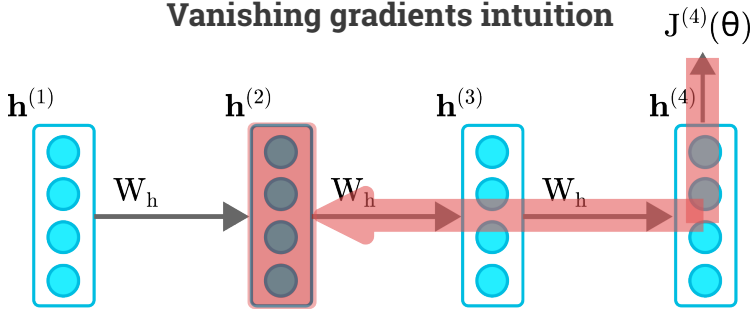
1. Vanishing gradients
2. Exploding gradients

Vanishing gradients intuition



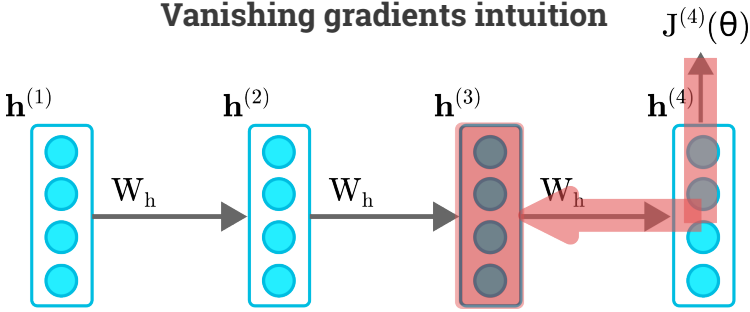
$$\frac{\partial J^{(4)}}{\partial \boldsymbol{h}^{(1)}} = ??$$

Vanishing gradients intuition



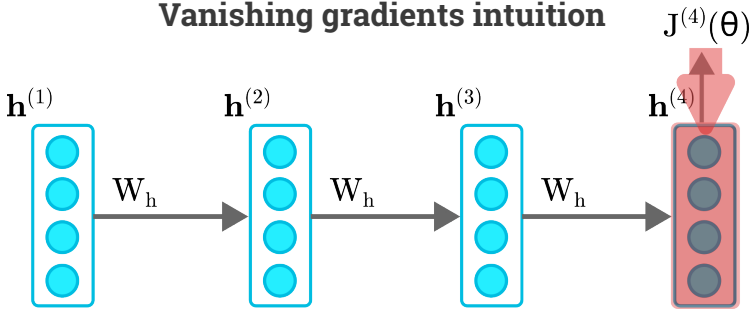
$$\frac{\partial J^{(4)}}{\partial \boldsymbol{h}^{(1)}} = \frac{\partial \boldsymbol{h}^{(2)}}{\partial \boldsymbol{h}^{(1)}} \times \frac{\partial J^{(4)}}{\partial \boldsymbol{h}^{(2)}}$$

Vanishing gradients intuition



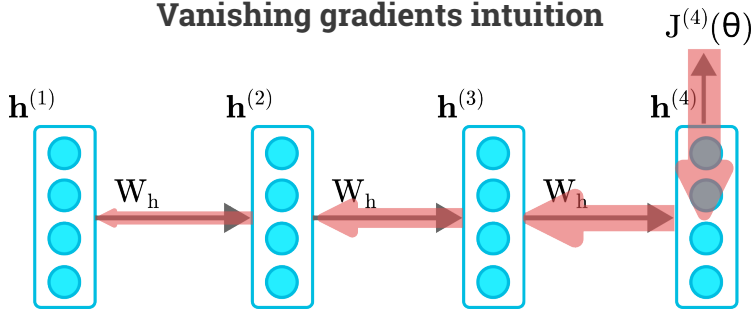
$$\frac{\partial J^{(4)}}{\partial \boldsymbol{h}^{(1)}} = \frac{\partial \boldsymbol{h}^{(2)}}{\partial \boldsymbol{h}^{(1)}} \times \frac{\partial \boldsymbol{h}^{(3)}}{\partial \boldsymbol{h}^{(2)}} \times \frac{\partial J^{(4)}}{\partial \boldsymbol{h}^{(3)}}$$

Vanishing gradients intuition



$$\frac{\partial J^{(4)}}{\partial \boldsymbol{h}^{(1)}} = \frac{\partial \boldsymbol{h}^{(2)}}{\partial \boldsymbol{h}^{(1)}} \times \frac{\partial \boldsymbol{h}^{(3)}}{\partial \boldsymbol{h}^{(2)}} \times \frac{\partial \boldsymbol{h}^{(4)}}{\partial \boldsymbol{h}^{(3)}} \times \frac{\partial J^{(4)}}{\partial \boldsymbol{h}^{(4)}}$$

Vanishing gradients intuition



$$\frac{\partial J^{(4)}}{\partial \boldsymbol{h}^{(1)}} = \frac{\partial \boldsymbol{h}^{(2)}}{\partial \boldsymbol{h}^{(1)}} \times \frac{\partial \boldsymbol{h}^{(3)}}{\partial \boldsymbol{h}^{(2)}} \times \frac{\partial \boldsymbol{h}^{(4)}}{\partial \boldsymbol{h}^{(3)}} \times \frac{\partial J^{(4)}}{\partial \boldsymbol{h}^{(4)}}$$

What if the gradients are small?

Vanishing gradients
Gradients w.r.t. the first tokens become tiny

Vanishing gradients proof sketch

Recall simplified model $\boldsymbol{h}^{(t)} = W_h \boldsymbol{h}^{(t-1)}, \quad J^{(t)}(\theta) = \text{CE}(\boldsymbol{h}^{(t)}, y^{*(t)})$

- Consider loss $J^{(i)}(\theta)$ with respect to hidden state $\boldsymbol{h}^{(j)}$ for some previous time step $j < i$. Let $l = i - j$

$$\frac{\partial J^{(i)}(\theta)}{\partial \boldsymbol{h}^{(j)}} = \frac{\partial J^{(i)}(\theta)}{\partial \boldsymbol{h}^{(i)}} \Big(\prod_{j < t \leq i} \frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{h}^{(t-1)}} \Big)$$

Vanishing gradients proof sketch

Recall simplified model $\boldsymbol{h}^{(t)} = W_h \boldsymbol{h}^{(t-1)}, \quad J^{(t)}(\theta) = \text{CE}(\boldsymbol{h}^{(t)}, y^{*(t)})$

- Consider loss $J^{(i)}(\theta)$ with respect to hidden state $\boldsymbol{h}^{(j)}$ for some previous time step $j < i$. Let $l = i - j$

$$\frac{\partial J^{(i)}(\theta)}{\partial \boldsymbol{h}^{(j)}} = \frac{\partial J^{(i)}(\theta)}{\partial \boldsymbol{h}^{(i)}} \Big(\prod_{j<t\leq i} \frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{h}^{(t-1)}} \Big)$$

$$= \frac{\partial J^{(i)}(\theta)}{\partial \boldsymbol{h}^{(i)}} \Big(\prod_{j<t\leq i} W_h \Big)$$

Vanishing gradients proof sketch

Recall simplified model $\boldsymbol{h}^{(t)} = W_h \boldsymbol{h}^{(t-1)}, \quad J^{(t)}(\theta) = \text{CE}(\boldsymbol{h}^{(t)}, y^{*(t)})$

- Consider loss $J^{(i)}(\theta)$ with respect to hidden state $\boldsymbol{h}^{(j)}$ for some previous time step $j < i$. Let $l = i - j$

$$\frac{\partial J^{(i)}(\theta)}{\partial \boldsymbol{h}^{(j)}} = \frac{\partial J^{(i)}(\theta)}{\partial \boldsymbol{h}^{(i)}} \Big(\prod_{j<t\leq i} \frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{h}^{(t-1)}} \Big)$$

$$= \frac{\partial J^{(i)}(\theta)}{\partial \boldsymbol{h}^{(i)}} \Big(\prod_{j<t\leq i} W_h \Big) = \frac{\partial J^{(i)}(\theta)}{\partial \boldsymbol{h}^{(i)}} W_h^l$$

If W_h "small", this term becomes minuscule as l grows

Vanishing gradients proof sketch

Problem: W_h^l for large l

- Consider the eigendecomposition of W_h, assume all $\lambda_i < 1$

$$\text{eigenvalues } \lambda_1, \lambda_2, \ldots, \lambda_n < 1$$
$$\text{eigenvectors } \boldsymbol{q}_1, \boldsymbol{q}_2, \ldots, \boldsymbol{q}_n$$

- We can write $\frac{\partial J^{(i)}(\theta)}{\partial \boldsymbol{h}^{(i)}} W_h^l$ using eigendecomposition of W_h

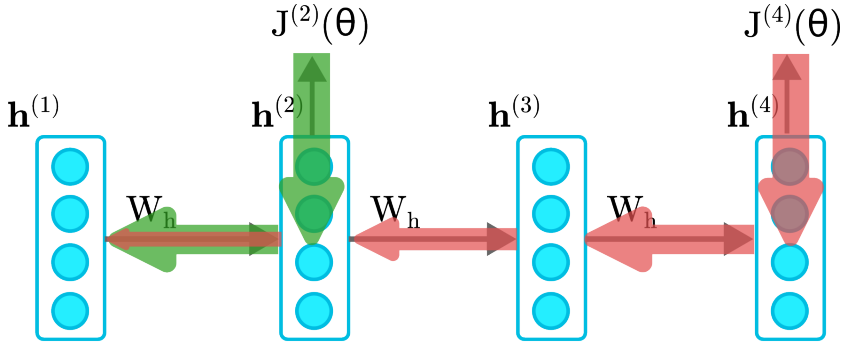
$$\frac{\partial J^{(i)}(\theta)}{\partial \boldsymbol{h}^{(i)}} W_h^l = \sum_{i=1}^{n} c_i \lambda_i^l \boldsymbol{q}_i \approx \mathbf{0} \text{ for large } l$$

Approaches 0 as l grows, so gradient vanishes

- This same phenomenon occurs with more realistic RNN models!

"On the difficulty of training recurrent neural networks", Pascanu et al, 2013.

Vanishing gradients: long term dependencies



Long term dependencies are impossible to model, because the gradient from far away tokens is much smaller than from near-by tokens

Vanishing gradients: an example

LM – Predict the next word: *After leaving work, he went to pick up his daughter from school. They had planned to go to the pet shop to buy a hamster and drove through town looking for one she liked. Eventually they found it, but before heading back home he had to rush back to his* _____

- To make correct prediction, need to model dependency between two word which are >20 tokens apart
- If long term gradients are small, they provide little signal compared to the short term gradients. Learning long-term dependencies becomes very slow.

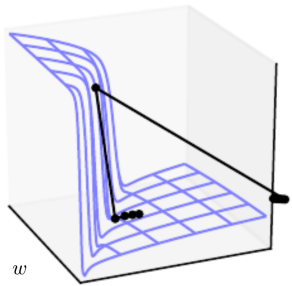
Vanishing gradients: an example

LM – Predict the next word: *After leaving work, he went to pick up his daughter from school. They had planned to go to the pet shop to buy a hamster and drove through town looking for one she liked. Eventually they found it, but before heading back home he had to rush back to his* _____

- To make correct prediction, need to model dependency between two word which are >20 tokens apart
- If long term gradients are small, they provide little signal compared to the short term gradients. Learning long-term dependencies becomes very slow.

LM – Predict the next word: *After leaving work, he went to pick up his daughter from school. They had planned to go to the pet shop to buy a hamster and drove through town looking for one she liked. Eventually they found it, but before heading back home he had to rush back to his work*

Exploding gradients



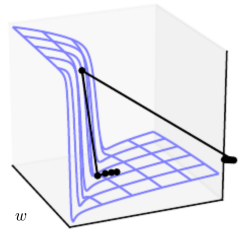
- Similarly if the eigenvalues of W_h are > 1, gradients become too big
- Exploding gradients can increase the loss instead of decreasing it
- In the worst case it can result in weights becoming `inf` or `NaN`

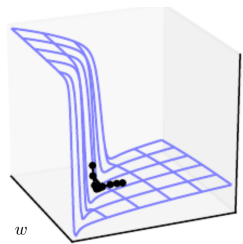
A solution for exploding gradients: gradient clipping

If gradients are too large, scale them down

1. set $\hat{g} \leftarrow \nabla J(\theta)$
2. if $\|\hat{g}\| \geq \tau$ for some threshold $\tau > 0$
3. set $\hat{g} \leftarrow \frac{\tau}{\|\hat{g}\|}\hat{g}$
4. end if

- the update direction remains the same, but the step-size is reduced
- clipping essentially solves the exploding gradients problem





Solutions for vanishing gradients?

- Vanilla RNNs cannot learn long-range dependencies because they don't have a memory

$$\boldsymbol{h}^{(t)} = \sigma\left(W_h \boldsymbol{h}^{(t-1)} + W_e \boldsymbol{e}^{(t)}\right)$$

The hidden state gets constantly rewritten

Solutions:

- RNNs may be adapted with separate memory mechanisms: LSTMs, GRUs. State of the art NLP models in 2010s.
- Create direct connections between far-away tokens: attention, residual connections